

מודול 5 — איסוף מידע

חומר לימוד

1 יחידות

תוכן העניינים

1. מודול 5 — איסוף מידע (Reconnaissance)

הכול בגלילה אחת: כל החומר של המודול בקובץ אחד — חומר לימוד, תרגילים, תרגול ופתרונות, ברצף.

🎬 מודול מעשי! אחרי הנושאים יש “ 🎬 הדגמה — נסה בעצמך”. תריץ אתר חברה מדומה (recon_target.py — בלי התקנות, פייתון בלבד) ותאסוף עליו מידע כמו תוקף אמיתי: קוד מקור, robots.txt, כותרות HTTP וקובץ גיבוי חשוף — 4 דגלים באיסוף מידע בלבד. אחרי כל הדגמה: 🧠 בדוק את עצמך.

מודול 5 — איסוף מידע (Reconnaissance)

מטרות המודול: לשלוט בשלב הראשון והחשוב ביותר בבדיקת חדירות — איסוף מידע על היעד. נלמד את ההבחנה בין איסוף פסיבי לאקטיבי, אימות היעד וגבולות ההרשאה (Scope), OSINT על אנשים ומיילים, מיפוי דומיינים ותת-דומיינים, זיהוי טכנולוגיות אתר, חיפוש מתקדם בגוגל (Google Dorking), והיכרות ראשונית עם Burp Suite.

למה זה הכי חשוב? ככל שאיסוף המידע יסודי יותר, כך גדל הסיכוי למצוא חולשה ולנצל אותה. בודקי חדירות מנוסים משקיעים חלק ניכר מהזמן דווקא כאן — “מי שמכיר את היעד טוב יותר, מנצח”.

קבצים: README.md · missions.md · solutions.md · practice.md · slides.md.

5.1 פסיבי מול אקטיבי (Passive vs. Active)

איסוף מידע מתחלק לשני סוגים, לפי מידת המגע עם היעד:

מגע עם היעד	פסיבי (Passive)	אקטיבי (Active)
מקורות	גוגל, LinkedIn, דלף מידע, רשומות ציבוריות	סריקה, חיבור לשירותים, בקשות לשרת
סיכוי להתגלות	נמוך מאוד	גבוה יותר
דוגמה	חיפוש עובד ב-LinkedIn	הרצת nmap על השרת

💡 **סדר העבודה:** תמיד מתחילים בפסיבי (שקט, לא מותיר עקבות), ורק אחר כך עוברים לאקטיבי. את השלב האקטיבי המלא (סריקה) נרחיב במודול 6.

🎬 הדגמה — נסה בעצמך: פסיבי מול אקטיבי, בפועל

נראה את ההבדל במו עינינו — על דומיין ציבורי לתרגול (example.com , שמור לדוגמאות).

איסוף פסיבי (מתשאלים שרתים ציבוריים, לא נוגעים ביעד):

```
whois example.com | head -20      # מי הבעלים, מתי נרשם
dig example.com A +short          # IP-כתובת ה
dig example.com MX +short         # שרתי הדואר (שימושי לפישינג)
```

👁️ קיבלת מידע על היעד — והוא **לא יודע** שחקרת אותו. (אין אינטרנט? זה בסדר, נמשיך למטרה מקומית).

איסוף אקטיבי (מגע ישיר — היעד רואה אותך):

```
ping -c 2 example.com             # חבילות ישירות ליעד
```

👁️ ה- ping נשלח **אל** היעד — פעולה אקטיבית שמותירה עקבות בלוגים שלו.

🎯 **מה קרה כאן:** dig / whois שאלו שרתי צד-שלישי (פסיבי, שקט); ping נגע ביעד עצמו (אקטיבי, גלוי). לכן תמיד פסיבי קודם.

🧠 **בדוק את עצמך:** איזו פעולה היא **פסיבית**? - א) הרצת nmap על שרת היעד - ב) whois על הדומיין (שאייתה למאגר רישום ציבורי) - ג) ping ליעד

✅ **תשובה: ב** — whois שואל מאגר ציבורי, בלי לגעת ביעד. nmap ו-ping הם מגע ישיר = אקטיבי.

שני צירים של איסוף

- **Physical / Social** — סיור פיזי במקום, הנדסה חברתית (Social Engineering), תמונות לוויין/רחפן, איתור עובדים. הקורס אינו מתמקד בזה אך חשוב להכיר את הקיום.
- **Web / Host** — מיקוד הקורס: מידע על הדומיין, השרת, הטכנולוגיות והמיילים.

5.2 אימות היעד וגבולות ההרשאה (Target Validation & Scope)

לפני כל פעולה — ודא שאתה תוקף את היעד הנכון ובתוך גבולות ההרשאה.

למה זה קריטי?

תיעוד אמיתי: בודק חדירות הזין כתובת IP שגויה (טעות הקלדה של ספרה אחת) ותקף מערכת של גורם שלישי לגמרי לא מעורב. טעות כזו היא עבירה פלילית וחושפת אותך לתביעה.

Scope — מה מותר ומה אסור

בכל תוכנית Bug Bounty (למשל דרך פלטפורמת Bugcrowd או HackerOne) מוגדר: In-Scope — הנכסים שמותר לתקוף (למשל tesla.com וכל תת-דומיינים שלו). Out-of-Scope — נכסים אסורים במפורש (למשל דומיינים של חברות שנרכשו, feedback.tesla.com). Rules of Engagement (RoE) — כללי ההתקשרות: אילו סוגי תקיפה מותרים, שעות, קצב וכו'.

⚠ חוק ברזל: קרא את ה-Scope וה-RoE בכל פעם מחדש לפני שתתחיל — היעדים בתוכנית משתנים עם הזמן. מה שהיה In-Scope אתמול עשוי להיות מחוץ לתחום היום.

5.3 איתור כתובות מייל ו-OSINT על אנשים

מיילים של עובדים הם נכס יקר — הם משמשים לפשינג, ל-Password Spraying ולזיהוי דפוסי התארגנות. המטרה: לגלות את תבנית המייל של הארגון (למשל firstname.lastname@company.com).

כלים נפוצים לאיתור מיילים

כלי	תפקיד
hunter.io	חיפוש מיילים לפי דומיין; חושף את תבנית המייל של הארגון
phonebook.cz	מפרט כתובות מייל, דומיינים ו-URLs לדומיין נתון
clearbit (תוסף Chrome)	שילוב ב-Gmail; שולף מיילים ותפקידים של עובדים
voilanorbert.com	איתור מייל של אדם ספציפי

אימות מייל (Email Verification)

לאחר שמצאת מייל — בדוק שהוא פעיל לפני שתסתמך עליו: hippo / emailhippo, email-checker — בודקים אם המייל תקין וקיים. - haveibeenpwned (HIBP) — בודק אם המייל הופיע בדלף מידע (Data Breach). - תכונת “שכחתי סיסמה” (Forgot Password) בשירותים כמו Gmail — יכולה לאשר שחשבון קיים ואף לחשוף מייל חלופי חלקי או ספרות אחרונות של טלפון, המסייעים לקשר בין חשבונות.

הרעיון: לא מסתפקים בכלי אחד. משלבים מספר מקורות (מיילים + דלף מידע + ניתוח דפוסים) כדי לבנות תמונה מלאה של היעד ולקשר בין זהויות.

5.4 דלף מידע (Data Breaches)

דלף מידע הוא אחד המקורות היעילים ביותר להשגת מידע רגיש — ובמיוחד **פרטי התחברות (Credentials)**.

- פריצות לחברות גדולות (כמו Home Depot, Equifax, LinkedIn) הובילו לחשיפת מיליוני זוגות משתמש-סיסמה.
- במקום להסתמך רק על סריקות למציאת חולשות, אפשר להשתמש ב**פרטי גישה שדלפו** כדי להיכנס למערכת ישירות, או כבסיס ל-Password Spraying.
- כלים ומאגרים: **HaveIBeenPwned**, ומאגרי Breach ייעודיים.

⚠ שימוש בפרטי גישה שדלפו מותר **אך ורק** בתוך ה-Scope של בדיקה מורשית.

5.5 תת-דומיינים (Subdomains)

תת-דומיין (Subdomain) הוא הרחבה של הדומיין הראשי. לדוגמה עבור `tesla.com` : `dev.tesla.com` — סביבת פיתוח - `login.tesla.com` — עמוד התחברות - `internal.tesla.com` — מערכות פנימיות

למה תת-דומיינים חשובים כל כך?

1. **הזדמנויות חדשות** — לעיתים חושפים מערכות שלא אמורות להיות גלויות (סביבות פיתוח, בדיקות).
2. **הרחבת משטח התקיפה** — במקום להתמקד רק בדומיין הראשי, בוחנים את כל התשתית.
3. **חולשות** — מערכות בתת-דומיינים (בעיקר `dev/test`) לרוב מאובטחות פחות.

כלים לגילוי תת-דומיינים

- **Sublist3r** — כלי קוד-פתוח פופולרי לגילוי תת-דומיינים.
- **Google Dorking**, **crt.sh** (Certificate Transparency), **Amass** (OWASP).

```
sudo apt install sublist3r
# או ידנית מ-GitHub:
sudo apt install python3 python3-pip -y
git clone https://github.com/about131a/Sublist3r.git
cd Sublist3r
pip install -r requirements.txt
python3 sublist3r.py -d tesla.com
```

- -d מציין את הדומיין לסריקה.

5.6 זיהוי טכנולוגיות האתר (Identifying Website Technologies)

כל אתר בנוי על מחסנית טכנולוגיות (Tech Stack) — שרת web, שפת תכנות, CMS, Framework ועוד. זיהוי מצמצם את חיפוש החולשות ומכוון לניצול הנכון.

כלים

כלי	מה מזהה
builtwith.com	פירוט מלא: Frameworks, שפות, אנליטיקס, CDN, אמצעי תשלום
Wappalizer (תוסף)	זיהוי מהיר של הטכנולוגיות בזמן גלישה
whatweb (Kali)	זיהוי טכנולוגיות משורת הפקודה

מה מחפשים?

- **שרת** Apache / Nginx / IIS — **Web וגרסתו** (גרסה ישנה = חולשה ידועה).
- **שפה ו-Framework** — PHP, Laravel, React, Bootstrap.
- **CMS** — WordPress, Drupal, Joomla (לכל אחד חולשות אופייניות).
- **שירותים נלווים** — CDN, מערכת תשלומים (Stripe), אנליטיקס.

דוגמה: אם [builtwith](https://builtwith.com) מגלה שהאתר מריץ Drupal ישן — נחפש Exploit ידוע ל-Drupal באותה גרסה במודול 7 (ניצול).

🎬 הדגמה — נסה בעצמך: זהה את הטכנולוגיה מהכותרות 🚩

כותרות ה-HTTP של שרת מלשינות עליו. נפעיל את אתר ACME ונחלץ את מחסנית הטכנולוגיות.

צעד 1 — טרמינל 1: הפעל את המטרה (`recon_target.py`):

```
python3 recon_target.py
```

👁️ האתר עולה על `http://127.0.0.1:8000` . השאר רץ.

צעד 2 — טרמינל 2: קרא רק את הכותרות (`-I` , `HEAD` , בלי גוף):

```
curl -sI 127.0.0.1:8000
```

👁️ חפש: `Server: Apache/2.4.49` — שרת וגרסה מדויקת. גרסה זו פגיעה (Path Traversal ידוע!) → יעד למודול 7. - `X-Powered-By: PHP/5.6.40` — שפה וגרסה ישנה. - `X-Recon-Flag: flag{http_headers_leak_tech_stack}` — הדגל הראשון! 🎉

צעד 3 — כלי ייעודי (אם מותקן):

```
whatweb http://127.0.0.1:8000
```

👁️ whatweb מזהה את אותו מידע אוטומטית.

🎯 מה קרה כאן: בלי לתקוף כלום, למדת איזה שרת, שפה וגרסאות רצות. גרסה מדויקת = חיפוש Exploit ממוקד. לכן אדמינים אמורים להסתיר את כותרת ה-Server.

🧠 בדוק את עצמך: למה `Server: Apache/2.4.49` בכותרת מסוכן לארגון? - (א) זה חושף את הגרסה המדויקת, ותוקף יכול לחפש חולשה ידועה לאותה גרסה - (ב) זה מאט את האתר - (ג) זה חוסם את גוגל

✅ תשובה: א — גרסה מדויקת מאפשרת לתוקף לחפש Exploit מתאים בדיוק. מומלץ להסתיר/לזייף את הכותרת (`ServerTokens Prod`).

5.7 חיפוש מתקדם בגוגל (Google Dorking / Google Fu)

גוגל הוא כלי OSINT עוצמתי. אופרטורי חיפוש (Search Operators) ממקדים את התוצאות וחושפים מידע רגיש שנחשף בטעות.

אופרטור	פעולה	דוגמה
<code>site:</code>	הגבלה לדומיין מסוים	<code>site:tesla.com</code>
<code>filetype:</code>	סוג קובץ	<code>filetype:pdf site:tesla.com</code>
<code>intitle:</code>	מילה בכותרת הדף	<code>intitle:"index of"</code>
<code>inurl:</code>	מילה בכתובת	<code>inurl:admin</code>
<code>intext:</code>	מילה בגוף הדף	<code>intext:"password"</code>
-	החרגה	<code>site:tesla.com -www</code>
" "	ביטוי מדויק	<code>"internal use only"</code>

שילוב אופרטורים חושף דפי ניהול, קבצים חשופים ומסמכים פנימיים. אתר תרגול: **Google** 
Hacking Database (GHDB) של Exploit-DB מרכז "Dorks" מוכנים.

Google Dorking על יעד הוא איסוף **פסיבי** — אך פעולה על הממצאים (כניסה לדף ניהול) עשויה 
 כבר להיות אקטיבית ובתוך ה-Scope בלבד.

5.8 היכרות ראשונית עם Burp Suite

Burp Suite הוא הכלי המרכזי לבדיקת אבטחת אפליקציות Web. הוא פועל כ-**Proxy** — יושב בין הדפדפן לשרת ומאפשר לצפות ולערוך את כל התעבורה (Requests ו-Responses).

מה Burp מאפשר?

- **לראות** כל בקשה שהדפדפן שולח וכל תשובה מהשרת.
- **לערוך** בקשות תוך כדי מעוף — לשנות פרמטרים, כותרות, ערכים.
- בסיס לתקיפות Web מתקדמות (נלמד בהמשך): Repeater, Intruder, Decoder.

הקמה בסיסית (Kali)

1. פתח **Burp Suite** (מותקן מראש ב-Kali) ← **Start** ← **Use Burp defaults** ← **Temporary Project**
Burp.

2. הגדר את הדפדפן (Firefox) לעבוד דרך ה-Proxy:

◦ **Manual proxy** ← **Network Settings** ← **Settings** ← Firefox.

◦ Port , 127.0.0.1 HTTP Proxy: 8080 (אותו הדבר לכל הפרוטוקולים).

3. להאזנה לתעבורה מוצפנת (HTTPS) — התקן את **תעודת ה-CA של Burp** בדפדפן.

💡 חלופה נוחה: התוסף **FoxyProxy** ל-Firefox מאפשר להדליק/לכבות את ה-Proxy בלחיצה אחת.

5.9 DNS ו-WHOIS — תשתית הדומיין

לפני שתוקפים שרת, כדאי להבין את **תשתית הדומיין**: מי הבעלים, אילו כתובות IP, ואילו שרתי דואר/שמות. כל אלה **פסיביים** (מתשאלים שרתי DNS ציבוריים, לא את היעד עצמו).

WHOIS — פרטי רישום הדומיין

```
whois tesla.com
```

ולעיתים אימייל/טלפון, (NS) שרתי שמות, Registrar, חושף: בעל הדומיין, תאריך רישום #

רשומות DNS — dig / nslookup / host

(של הדומיין IP כתובת) A רשומת:

```
dig tesla.com A +short
```

(שרתי דואר — שימושי לפשינג) MX רשומת:

```
dig tesla.com MX +short
```

(חושף ספקי דואר וענן — SPF/DKIM לרוב) TXT רשומת:

```
dig tesla.com TXT +short
```

ציבורי DNS כל הרשומות דרך שרת:

```
dig @8.8.8.8 tesla.com ANY
```

חלופות נפוצות:

```
nslookup tesla.com
```

שאילתה בסיסית

```
host -t ns tesla.com
```

שרתי השמות (Name Servers)

Zone Transfer (ניסיון — לרוב חסום, אך קטלני כשעובד)

מקבלים את כל רשומות הדומיין בבת אחת — AXFR מוגדר שגוי ומאפשר DNS אם שרת:

```
dig axfr @ns1.target.com target.com
```

🎯 **Zone Transfer** מוצלח = מפת כל תת-הדומיינים והשרתים במכה אחת. נדיר בארגונים מודרניים,

5.10 דוגמאות מעשיות — שרשרת איסוף פסיבי

הנה **תרחיש מלא** של איסוף פסיבי על דומיין (כל הפקודות פסיביות/חוקיות לתרגול):

```
# 1. פרטי רישום ותשתית
whois example.com
dig example.com A MX TXT +short

# 2. theHarvester (איסוף מיילים, שמות ותת-דומיינים במכה אחת):
theHarvester -d example.com -b bing,duckduckgo,crtsh
# -d = דומיין · -b = (מנועי חיפוש/מאגרים)
# "missing API key" אך אז יופיעו שגיאות, -b all טיפ: אפשר גם
# למקורות שדורשים מפתח – זה תקין, פשוט התעלם מהן.

# 3. תת-דומיינים ייעודי:
sublist3r -d example.com
# curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '[][.name_value]'
או ללא התקנה

# 4. (פסיבי-קליל – whatweb Web-זיהוי טכנולוגיות שרת ה):
whatweb https://example.com

# 5. (חושף שרת, טכנולוגיה, הפניות) HTTP כותרות:
curl -sI https://example.com
# (HEAD) כותרות בלבד -I · שקט -s

# 6. Google Dork (בדפדפן) ממוקד:
# site:example.com filetype:pdf
# site:example.com inurl:admin
```

הפלט הטיפוסי מזין את השלבים הבאים: - מיילים → פשינג / Password Spraying (מודול 8, 15). - תת-דומיין .dev → יעד רך לסריקה אקטיבית (מודול 6). - גרסת שרת ישנה → חיפוש Exploit (מודול 7).

💡 **טיפ מקצועי:** שמור את כל הפלט לקובץ (recon.html ... theHarvester). כל פיסת מידע היא נקודת פתיחה אפשרית — ותיעוד מסודר עכשיו חוסך שעות בהמשך ובכתיבת הדוח (מודול 16).

5.11 מיני-CTF: אסוף מידע על אתר ACME

עכשיו הכול ביחד. אתר ACME (`recon_target.py`) מחביא 4 דגלים, כולם מתגלים באיסוף מידע בלבד — בלי "לתקוף". זו בדיוק העבודה של שלב ה-Recon.

🎬 הדגמה — נסה בעצמך: צוד את 4 הדגלים

הכנה — טרמינל 1:

```
python3 recon_target.py          # השאר רץ - http://127.0.0.1:8000
```

דגל 1 — קוד המקור. תוקפים תמיד קוראים את מקור הדף (הערות מפתחים חושפות סודות):

```
curl -s 127.0.0.1:8000/ | grep -i "flag\|TODO\|admin"
```

👁️ הערת 👁️ <!-- TODO ... flag{view_source_reveals_secrets} --> + רמז לנתיב /admin-panel/

דגל 2 — כותרות HTTP:

```
curl -sI 127.0.0.1:8000 | grep -i flag
```

👁️ flag{http_headers_leak_tech_stack} (וגם הגרסה הפגיעה של השרת).

דגל 3 — `robots.txt`. הקובץ שאמור "להסתיר" נתיבים — דווקא חושף אותם:

```
curl -s 127.0.0.1:8000/robots.txt
```

👁️ /admin-panel/ Disallow: /backup/ . עכשיו בקר בפאנל:

```
curl -s 127.0.0.1:8000/admin-panel/ | grep -o 'flag{[^}]*}'
```

👁️ . flag{robots_txt_leaks_hidden_paths}

דגל 4 — קובץ גיבוי חשופ. robots הסגיר גם את /backup/ :

```
curl -s 127.0.0.1:8000/backup/config.bak
```

👁️ DB אמיתית + flag{exposed_backup_files_are_gold} .

🎯 **מה קרה כאן:** ב-4 בקשות `curl` תמימות (כולן GET לקריאה בלבד) בנית תמונה שלמה: טכנולוגיה, גרסה פגיעה, נתיבים חבויים וסיסמת מסד נתונים — **כל זה לפני שתקפת משהו**. זה הכוח של Recon יסודי.

🏆 **פעילות כיתתית:** מי אוסף את כל 4 הדגלים ראשון? ואז — דיון: איזו הגדרה הייתה מונעת כל דליפה? (הסתרת Server header, אי-הישענות על robots להגנה, הסרת קובצי backup, ניקוי הערות בקוד).

🧠 **בדוק את עצמך:** מה הטעות ב"הסתרת" נתיב רק דרך `robots.txt` ? - א) `robots.txt` מאט את השרת - ב) הקובץ ציבורי וקריא לכולם — הוא בעצם **מפרסם** את הנתיבים שרצית להסתיר - ג) גוגל חוסם את האתר

✅ **תשובה: ב** — `robots.txt` רק מבקש ממנועי חיפוש לא לאנדקס; הוא אינו הגנה. תוקף קורא אותו ראשון כדי לגלות נתיבים "מוסתרים".

סיכום המודול

- **פסיבי לפני אקטיבי** — מתחילים בשקט, בלי מגע עם היעד.
- **אמת את היעד וקרא את ה-Scope** בכל פעם — טעות כאן היא עבירה פלילית.
- **OSINT על מיילים ואנשים** — `hunter.io`, `clearbit`, `HIBP`; מטרה: תבנית המייל.
- **דלף מידע** מספק פרטי גישה אמיתיים — יעיל מאוד, אך רק בתוך ה-Scope.
- **תת-דומיינים** (`Sublist3r`) מרחיבים את משטח התקיפה וחושפים מערכות פגיעות.
- **זיהוי טכנולוגיות** (`builtwith`, `Wappalyzer`) ממקד את חיפוש החולשות.
- **Google Dorking** חושף מידע רגיש עם אופרטורים ממוקדים.
- **DNS ו-WHOIS** (`whois`, `dig`, `nslookup`) חושפים את תשתית הדומיין; `Zone Transfer` = מפה מלאה כשעובד.
- **theHarvester** מרכז מיילים, שמות ותת-דומיינים בפקודה אחת; `curl -sI` ו-`whatweb` לזיהוי מהיר.
- **Burp Suite** הוא ה-Proxy המרכזי לבדיקת תעבורת Web.

מה הלאה?

➡ תרגילים — `missions.md` · פתרונות — `solutions.md` · תרגול ותרחישים — `practice.md` ➡
מודול 6 — סריקה, מיפוי והערכת חולשות

⬅ חזרה למפת הקורס